Neural Code Generation with Deep Sequence Models

Hiba Qutbuddin Habib **Hibatallah Belhajali**

KTH Royal Institute of Technology
Deep Learning in Data Science, DD2424
{hibaqh, hibbel}@kth.se

Abstract

This project investigates character-level code generation using recurrent neural networks trained on Python functions from the CodeSearchNet dataset. Three model architectures were evaluated: a vanilla RNN, a single-layer LSTM, and a two-layer LSTM. The models were trained to predict the next character in a sequence, using temperature-scaled and nucleus sampling for generation.

A grid search was conducted to find optimal hyperparameters. All models performed best with 256 hidden units, batch size 32, and learning rate 0.001. These models were then fine-tuned on the full dataset. Training loss curves showed that the two-layer LSTM achieved the most stable convergence and lowest loss, while the RNN struggled with instability and higher error.

Generated text was evaluated both quantitatively and qualitatively. N-gram overlap showed similar performance across models, with RNN slightly better on bigrams, and LSTM models showing more stable trigram patterns. Qualitative evaluation included spell checking and manual inspection. The two-layer LSTM produced the most coherent and syntactically valid code, improving spelling accuracy from 80% to over 93% after fine-tuning.

Overall, deeper LSTM models proved better suited for character-level code generation. Although they may underperform in simple n-gram metrics, they generate more fluent and structurally sound code. These findings support the use of multi-layer LSTMs for programming language modeling at the character level.

1 Introduction

This project explores character-level Python code generation using deep recurrent models. Specifically, three architectures are compared: a vanilla Recurrent Neural Network (RNN), a 1-layer Long Short-Term Memory (LSTM), and a 2-layer LSTM. The motivation stems from the growing demand for automated coding tools—driven by systems like GitHub Copilot and ChatGPT—which rely on models capable of generating syntactically correct and semantically meaningful source code.

Unlike natural language, code contains strict grammar rules and long-range dependencies. Effective code generation requires capturing both local patterns (e.g., punctuation and indentation) and global structure (e.g., control flow, function blocks). This makes character-level modeling especially challenging yet powerful, as it avoids the complexities of tokenization.

To address these challenges, models were trained to predict the next character in a sequence using the CodeSearchNet Python dataset. A grid search over hyperparameters (hidden size, learning rate, batch size) identified optimal configurations. Text generation was evaluated using both temperature scaling and nucleus sampling to control diversity and coherence.

Results show that the 2-layer LSTM outperforms the others in training stability, syntactic quality, and spelling accuracy—achieving over 93% correctness after fine-tuning. While n-gram overlap was similar across models, the deeper LSTM produced more coherent code sequences overall.

This study demonstrates that deeper recurrent models significantly improve code generation at the character level and that advanced sampling techniques further enhance quality, suggesting a strong baseline for future work on generative code models.

2 Related Work

Character-level sequence models for code generation were popularized by Karpathy's `char-RNN` (1), which demonstrated that multi-layer LSTMs trained on raw character sequences can produce syntactically plausible outputs, even for source code. Since then, recurrent neural networks (RNNs), LSTMs, and GRUs have been widely applied to code modeling tasks such as autocompletion (2), language modeling (3), and code synthesis (4).

Several works have shown that LSTMs and GRUs outperform basic RNNs on code modeling tasks due to their ability to capture long-range dependencies (5). For example, Hellendoorn and Devanbu (6) found that LSTM-based language models for source code outperform traditional n-gram models on prediction accuracy, especially for structured code patterns.

Model depth is another factor explored in prior research. Van Scharrenburg (7) compared single- and two-layer LSTM/GRU models and found that deeper models generalized better on code completion tasks. Similar findings were observed in other domains, where deeper LSTM architectures led to improvements in sequence modeling (8).

Text generation methods such as temperature scaling and nucleus sampling are widely used to improve diversity in generated sequences. Temperature adjusts the sharpness of the output distribution (9), while nucleus sampling (top-p) was introduced by Holtzman et al. (10) as a method to sample from the most probable region of the output space, yielding fluent and diverse outputs. These techniques have been adapted to neural code generation to balance syntactic correctness with creativity.

In terms of evaluation, early models relied on metrics such as perplexity and BLEU to quantify generation quality (11). More recent work emphasizes syntax validity (e.g., parseability or compilation success) and functional correctness (e.g., test case accuracy). Tools like CodeBLEU (12) combine lexical, syntactic, and semantic features to better assess code generation quality.

This project builds on these foundations by comparing RNN and multi-layer LSTM architectures for character-level Python code generation, using temperature and nucleus sampling to study sampling effects, and evaluating both quantitatively (loss, n-grams) and qualitatively (syntax correctness).

3 Data

This project uses the Python subset of the CodeSearchNet dataset (13), containing Python functions extracted from GitHub repositories, each paired with documentation and metadata. Only the `func_code_string` field was used, as the goal was to train a character-level language model for code generation.

3.1 Processing Pipeline

The complete data pipeline is summarized in Figure 1. Before preprocessing, basic data exploration was performed to inspect structure and function lengths. As shown in Figure 2, most snippets are short, though some outliers exceed 50,000 characters.

To improve training efficiency, only functions under 5000 characters were kept. Empty code, comments, and docstrings were removed. The remaining code snippets were concatenated into one string, from which a character vocabulary was built and mapped to integer indices.

The resulting sequence was split into overlapping segments of 100 characters, each with the next character as target. These were batched and shuffled to enable parallel training and robust learning.

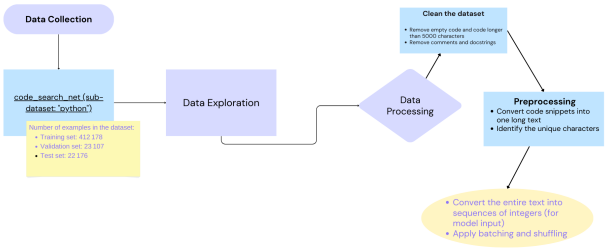


Figure 1: Data processing pipeline.

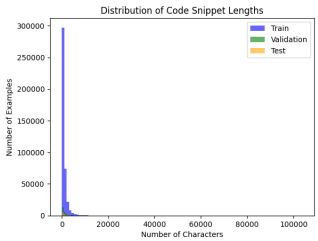


Figure 2: Distribution of code snippet lengths.

4 Methods

To solve the task of character-level Python code generation, a systematic approach was followed—from data collection to evaluation. An overview of the full pipeline is shown in Figure 3.

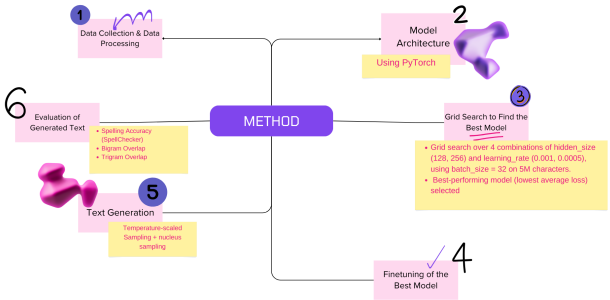


Figure 3: Overview of the method pipeline.

4.1 Model Architecture

The model, referred to as *CharRNN*, was implemented in PyTorch and consists of an embedding layer, a recurrent layer (RNN or LSTM), and a linear output layer. The embedding maps character indices to dense vectors, offering memory-efficient representations that can capture relationships between characters, unlike sparse one-hot encodings.

Three recurrent variants were tested: vanilla RNN, single-layer LSTM, and two-layer LSTM. LSTMs are better at preserving long-range dependencies, and stacking layers (as in LSTM-2) further improves sequence modeling, though with higher computational cost. The final linear layer maps hidden states to vocabulary logits for next-character prediction.

4.2 Training and Optimization

Training used `CrossEntropyLoss` and the Adam optimizer. Each input batch consisted of sequences of 100 characters, with the model predicting the next character at each step. Loss was backpropagated and parameters updated with `optimizer.step()`. The data was preprocessed into overlapping sequences, batched, and shuffled at each epoch to encourage generalization.

4.3 Hyperparameter Optimization

A grid search was conducted over hidden sizes (128, 256), learning rates (0.001, 0.0005), and batch sizes (32, 64). Each configuration was trained on a 5M character subset. The model with lowest average loss was selected and fine-tuned on the full dataset.

4.4 Text Generation and Evaluation

After training, code was generated using both temperature scaling and nucleus sampling, which control randomness and diversity. Evaluation was done quantitatively (training/validation loss, n-gram overlap) and qualitatively (spell checking, syntactic inspection). These metrics provided insight into both local structure and output coherence.

4.5 Method Justification

Character-level modeling was chosen due to Python's sensitivity to individual characters (e.g., colons, parentheses, indentation). Unlike token-level models, it avoids the complexity of handling rare or unknown tokens and enables direct training on raw code. This leads to a simpler pipeline and potentially greater generalization.

5 Experiments

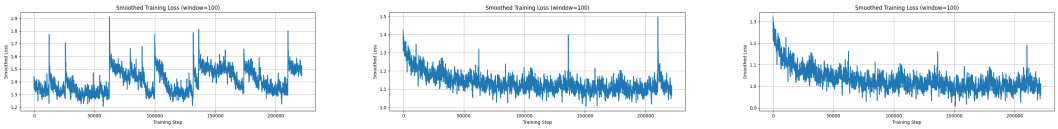
The experiments were conducted following the pipeline described in Section 4. The results indicate that the proposed method is effective for character-level code generation. Both quantitative and qualitative evaluations were carried out to assess model performance.

5.1 Quantitative Analysis

The quantitative evaluation examined training loss and the similarity between generated text and real code using n-gram overlap. All models were tuned using grid search, where the best configuration—256 hidden units, batch size 32, and learning rate 0.001—consistently yielded the lowest average loss: 1.56 (RNN), 1.61 (LSTM-1), and 1.52 (LSTM-2).

5.1.1 Training Loss

Figure 4 shows the smoothed training loss curves after fine-tuning. The two-layer LSTM (LSTM-2) demonstrated the most stable learning and lowest final loss. LSTM-1 showed moderate improvements, while the vanilla RNN exhibited high fluctuations and slower convergence, suggesting difficulty in modeling longer dependencies. These results confirm that LSTM-based models are more effective for sequence modeling at the character level.



(a) Loss RNN (b) Loss LSTM-1 (c) Loss LSTM-2

Figure 4: Smoothed training loss for the three models during fine-tuning.

5.1.2 N-gram Overlap

To evaluate how well each model captures real code patterns, bigram and trigram overlap with the test set was computed. Sampling was performed using nucleus sampling (p) and temperature (T), with $p = 0.9$ and $T = 1.0$ chosen for reporting due to their balanced output quality. As shown in Table 1, results were relatively close across models. RNN achieved the highest bigram overlap, while LSTM-1 and LSTM-2 performed similarly in trigram overlap, indicating comparable modeling of local structure. Full results across all sampling settings are available in Appendix D.

Table 1: Bigram and trigram overlap (%) between generated text and test data (T = 1.0, p = 0.9).

Model	Bigram (%)	Trigram (%)
RNN	38.64	22.09
LSTM-1	37.65	22.09
LSTM-2	36.82	20.76

5.2 Qualitative Analysis

In addition to quantitative metrics, a qualitative analysis was performed through manual inspection and automated spellchecking of generated text. The aim was to assess whether finetuning improved syntactic fluency and coherence across models.

5.2.1 Spelling Accuracy

Spelling accuracy was measured as the percentage of correctly spelled English words in the generated code, before and after fine-tuning. As shown in Table 2, the RNN and LSTM-1 models showed only marginal changes. However, the LSTM-2 model demonstrated a substantial improvement, increasing from 80.39% to 93.75%. This result indicates that deeper architectures may benefit more from additional training and are more capable of generating syntactically valid sequences. Additional spelling results across more settings can be found in Appendix E.

Table 2: Spelling accuracy (%) before and after finetuning using temperature = 1.0 and p = 0.9.

Model	Stage	Accuracy (%)
RNN	Before	81.48
	After	80.95
LSTM-1	Before	72.97
	After	72.73
LSTM-2	Before	80.39
	After	93.75

5.2.2 Syntactic and Semantic Quality

Manual inspection of generated samples confirmed the quantitative trends. The RNN model frequently produced incoherent and fragmented code, both before and after fine-tuning, lacking basic syntax such as valid function headers or meaningful variable names. LSTM-1 showed some improvements after fine-tuning, with better structure and recognizable control statements. The most significant gains were observed in LSTM-2, which generated well-structured functions containing appropriate Python syntax, including `try`, `for`, `return`, and semantically relevant names such as `get_buffer` and `execute`. This qualitative improvement aligns with spelling and n-gram metrics, confirming that deeper LSTM models are better suited for character-level code generation tasks. Representative generated samples for each model, both before and after fine-tuning, are provided in Appendix C.

5.3 Model Comparison Summary

The three models differ in depth and recurrence type: RNN uses a single vanilla recurrent layer, LSTM-1 adds gating mechanisms with one layer, while LSTM-2 stacks two LSTM layers for deeper

sequence modeling. LSTM-2 consistently outperformed the others in spelling accuracy, training loss, and syntactic structure due to its enhanced capacity to capture long-term dependencies. However, its bigram and trigram overlap was slightly lower, likely because deeper models generalize beyond local patterns rather than memorizing them. This tradeoff suggests LSTM-2 generates more coherent code, even if it deviates more from the training distribution.

6 Conclusion

This project investigated character-level Python code generation using recurrent neural networks. Three architectures were compared: vanilla RNN, single-layer LSTM, and two-layer LSTM. The two-layer LSTM consistently achieved the best performance in terms of training loss, spelling accuracy, and syntactic quality. Although n-gram overlap with test data was similar across models, the deeper LSTM showed clear advantages in producing coherent and structurally valid Python code, highlighting the importance of model depth for sequence modeling.

To further improve performance, future work could explore transformer-based models like GPT, integrate syntax-aware constraints during training, or fine-tune models on domain-specific codebases. Additionally, assessing functional correctness through automated execution and testing would offer a more rigorous evaluation of generated code.

Overall, the results show that relatively simple recurrent models can learn meaningful code structures, and that deeper architectures like LSTM-2 provide a strong foundation for more advanced generative systems.

Link to the code repository: https://github.com/HQUT/Deep-Learning-in-Data-Science-/blob/main/Project.ipynb

References

[1] A. Karpathy. The unreasonable effectiveness of recurrent neural networks. http://karpathy.github.io/2015/05/21/rnn-effectiveness/, 2015.

[2] M. White et al. Toward deep learning software repositories. In *MSR*, 2015. https://doi.org/10.1109/MSR.2015.35.

[3] H. K. Dam, T. Tran, and T. Pham. A deep language model for software code. *arXiv preprint arXiv:1608.02715*, 2016. https://arxiv.org/abs/1608.02715.

[4] W. Ling et al. Latent predictor networks for code generation. In *ACL*, 2016. https://aclanthology.org/P16-1154/.

[5] J. Chung et al. Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv:1412.3555*, 2014. https://arxiv.org/abs/1412.3555.

[6] V. J. Hellendoorn and P. Devanbu. Are deep neural networks the best choice for modeling source code? In *FSE*, 2017. https://doi.org/10.1145/3106237.3106286.

[7] E. van Scharrenburg. Code completion with recurrent neural networks. MSc Thesis, TU Delft, 2019. https://repository.tudelft.nl/islandora/object/uuid:996d60e8-7701-48f0-9c01-0aaf25604a57.

[8] I. Sutskever, O. Vinyals, and Q. Le. Sequence to sequence learning with neural networks. In *NeurIPS*, 2014. https://papers.nips.cc/paper_files/paper/2014/file/a14ac55a4f27472c5d894ec1c3c743d2-Paper.pdf.

[9] A. Fan, M. Lewis, and Y. Dauphin. Hierarchical neural story generation. In *ACL*, 2018. https://aclanthology.org/P18-1082/.

[10] A. Holtzman et al. The curious case of neural text degeneration. In *ICLR*, 2020. https://arxiv.org/abs/1904.09751.

[11] K. Papineni et al. BLEU: a method for automatic evaluation of machine translation. In *ACL*, 2002. https://aclanthology.org/P02-1040/.

[12] S. Ren et al. CodeBLEU: a metric for evaluating code generation. *arXiv:2009.10297*, 2020. https://arxiv.org/abs/2009.10297.

[13] Dataset https://huggingface.co/datasets/code-search-net/code_search_net.

A Sample Data Entries from CodeSearchNet

The following example illustrates the structure of the processed dataset used for training and evaluation. Each entry represents a Python function extracted from a GitHub repository, including metadata, source code, and annotations.

Example 1

- **Repository Name**: `cisco-sas/kitty`
- **File Path**: `kitty/data/data_manager.py`
- **Function Name**: `Table.update`
- **Language**: `Python`
- **Split Name**: `train`
- **Function Code URL**: https://github.com/cisco-sas/kitty/blob/cb0760989dcdfe079e43ac574d872d0b18953a32/kitty/data/data_manager.py#L331-L347

Function Code String

```python
def update(self, field_dict, where_clause=None):
    '''
    update db entry

    :param field_dict: dictionary of fields and values
    :param where_clause: where clause for the update
    '''
    query = '''
    UPDATE %s SET %s
    ''' % (
        self._name,
        ','.join('%s=:%s' % (k, k) for k in field_dict)
    )
    if where_clause:
        query += ' WHERE %s' % (where_clause)
    self._cursor.execute(query, field_dict)
    self._connection.commit()
```

Function Documentation String

```
update db entry

:param field_dict: dictionary of fields and values
:param where_clause: where clause for the update
```

Function Code Tokens (partial)

```
['def', 'update', '(', 'self', ',', 'field_dict', ',', 'where_clause', '=', 'None', ')', ':',
```

Function Documentation Tokens

```
['update', 'db', 'entry']
```

B Processed Code Examples After Data Cleaning

This appendix contains sample entries from the training dataset after cleaning and preprocessing. The functions shown below have had comments, docstrings, and irrelevant whitespace removed. Only code tokens relevant to model training remain.

Function ID: 300000

```
def show_slug_with_level(context, page, lang=None, fallback=True):
    if not lang:
        lang = context.get('lang', pages_settings.PAGE_DEFAULT_LANGUAGE)
    page = get_page_from_string_or_id(page, lang)
    if not page:
        return ''
    return {'content': page.slug_with_level(l
```

Function ID: 350000

```
def autoconf(self):
    ntpinfo = NTPinfo()
    ntpstats = ntpinfo.getHostOffsets(self._remoteHosts)
    return len(ntpstats) > 0
```

Function ID: 400000

```
def _force_text_recursive(data):
    if isinstance(data, list):
        ret = [_force_text_recursive(item) for item in data]
        if isinstance(data, ReturnList):
            return ReturnList(ret, serializer=data.serializer)
        return data
    elif isinstance
```

Additional Statistics:

- **Number of unique characters**: 3033
- **Character samples**: `['\x03', '\x04', '\t', '\n', '\r', '\x1b', '\x1f', ' ', '!', '"', '#', '$', '%', '&', '´', '(', ')', '*', '+', ',']`
- **Length of `train_as_int`**: 236,354,599
- **First 20 indices**: `[75, 76, 77, 7, 92, 87, 75, 72, 91, 76, 7, 15, 90, 76, 83, 77, 7, 19, 77, 80]`

C Generated Code Samples Before and After Finetuning

The following code snippets show raw outputs generated by the three model variants (RNN, LSTM-1, and LSTM-2) both before and after fine-tuning, using temperature = 1.0 and nucleus sampling with p = 0.9. These outputs illustrate the qualitative differences in code coherence and syntax.

RNN

Before Finetuning (Temp=1.0, p=0.9):

```
def clearnal_data(ax3=numpy.exit, exists=None):
    if evalibal_data.rel_noip().striptedCells(args):
        return transatems = offset
    return self._cum_initializer.link_import_call_tip_imply(
        'umodent-hetName', width_params
    )
    return t = 10
    return False

def _bed():
    return [0]

encoded_token = None
sub_checkpoints = []
return predictedActiveCells[0]
app_token = ['BIAME.sparse_send(distadiutsTreed)']
localSeq = ...
```

After Finetuning (Temp=1.0, p=0.9):

```
def _type():
    if find_compositimath == np.repled_types:
        if s:
            if not client_reshow == 'Impe':
                pass

def listen(self, checkine_types, path, camperview_abberly, *args):
    pass

def seq_id(val_entity=path):
    return args

return blockspect_templission[keys()]
raise self.idle_set_f.split(path=connection)

return self.parse("part_pathsize_probs_by_partition_d")
```

LSTM-1

Before Finetuning (Temp=1.0, p=0.9):

```
def batch(mst_status_name):
    exp = None
    fullPerimions = []
    log = False

sequence.config_apinalParse(help="--py")
contigs = {}

if motor(self.isplit_id):
    if time and alls:
        self.endport = pattern_args

self._get(numColumns)
```

```
def get(self, line, num_expanement_header):
    container.basestats(print())

if cation(self, bed, close_run['times']):
    if not unones:
        proto.is_field_path

try:
    attr = opts.add_address
    content = int(self.url)
except:
    pass
```

After Finetuning (Temp=1.0, p=0.9):

```
def file_extensions(self, f):
    return response

def get_buffer(self, value):
    set_tables = set()
    return discovers

def execute(iterable, path_helper, expected_type, fasta):
    method_message = val

    if self._token_date != obs_tokens:
        if not isinstance(tmp_template, source):
            raise PythonMouseException(
                "settype:␣ple␣await␣must␣comment␣password",
                decorator._setup_finish()
            )

    result = process_to_object(self, patterns, sequence_id, **kwargs)

def expand(plt, is_remove_convert_extension):
    state = None
```

LSTM-2

Before Finetuning (Temp=1.0, p=0.9):

```
def debug_parents(sorted_input):
    if the is None:
        return spec
    if i:
        p = opt.list(self.location)

    for t in "\t".join(dir, key=initialize):
        pass

from jcvi.find import newwindent
if not isinstance(dest_type, ExpectedType):
    account_axis = dir + length

assert exit_name == list

p.stats(id)

if len(key, ip, hash_name):
    pass

def get_to_method(self, size):
    pass

def stop(self, delta=None, length="frequency./3.0"):
```

```
    normalize = True
    to = "mail"

    for j in bed_options.loader.items():
        tm = ...
```

After Finetuning (Temp=1.0, p=0.9):

```
def property():
    if sys.version_id == 'validation':
        server = funct.get(service, url_param)
        return id

        text = url + '/node'
        filter = util.version
        docstring = settings_url

    try:
        return self.function(request, cls, *args, **kwargs)

        while len(self._click.query_code) <= 1:
            for face in content.items():
                call = col.copy()
                if records.items():
                    entry['char'] = self.chunk_size

            return value

        self.tr
```

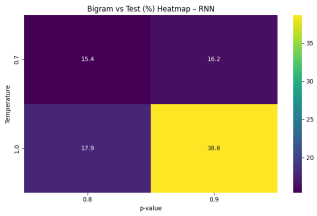
D Detailed Bigram and Trigram Overlap Results

This appendix presents detailed n-gram overlap results for all models across temperature and nucleus sampling settings. The second section presents visualizations.

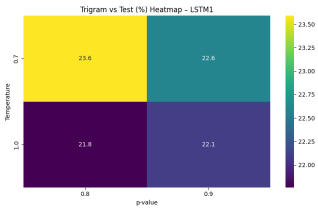
D.1 Bigram and Trigram Overlap (%) – Summary Table

Model	Temperature	p-value	Bigram (%)	Trigram (%)
RNN	0.7	0.8	15.42	12.62
RNN	0.7	0.9	16.25	13.46
RNN	1.0	0.8	17.91	13.29
RNN	1.0	0.9	38.64	22.09
LSTM1	0.7	0.8	36.98	23.59
LSTM1	0.7	0.9	37.98	22.59
LSTM1	1.0	0.8	36.82	21.76
LSTM1	1.0	0.9	37.65	22.09
LSTM2	0.7	0.8	31.51	18.94
LSTM2	0.7	0.9	33.17	20.60
LSTM2	1.0	0.8	34.66	19.77
LSTM2	1.0	0.9	36.82	20.76

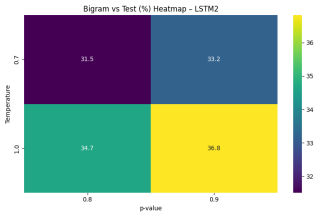
D.2 Visualizations



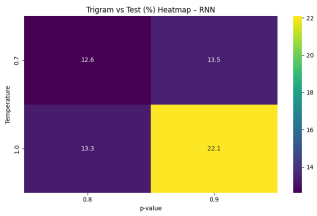
(a) RNN – Bigram Overlap



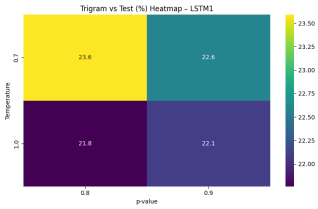
(b) LSTM1 – Bigram Overlap



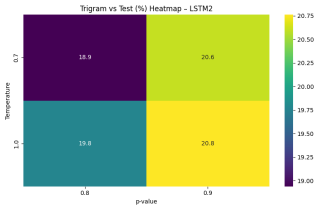
(c) LSTM2 – Bigram Overlap



(a) RNN – Trigram Overlap

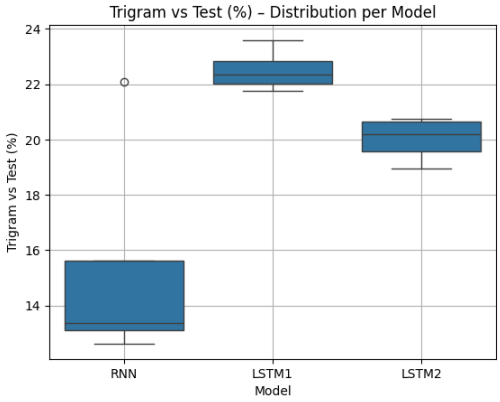


(b) LSTM1 – Trigram Overlap

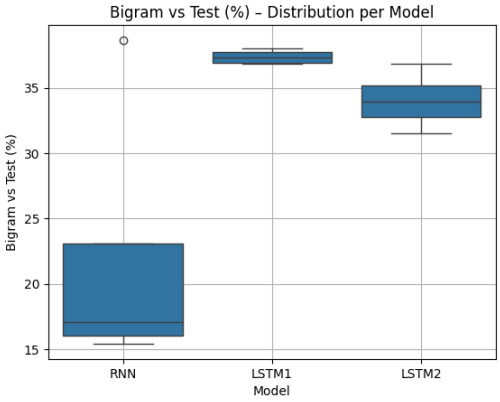


(c) LSTM2 – Trigram Overlap

These heatmaps show the overlap percentages for bigram and Trigram overlapp between generated text and test data for RNN, LSTM1 and LSTM2. Each cell corresponds to a combination of temperature and p-value, revealing how sampling parameters affect n-gram similarity.



(a) Trigram Overlap – All Models



(b) Bigram Overlap – All Models

These boxplots compare the n-gram overlap distributions across different models (RNN, LSTM1, LSTM2). The trigram plot shows that LSTM1 has the highest consistency and overlap. The bigram plot reflects similar trends, though LSTM2 shows slightly more spread.

E Spelling Accuracy Results

Table 3: Spelling Accuracy (%) by Model, Stage, Temperature and p-value

Model	Stage	Temperature	p-value	Accuracy (%)
RNN	Before	0.7	0.8	83.67
RNN	Before	0.7	0.9	86.96
RNN	Before	1.0	0.8	77.78
RNN	Before	1.0	0.9	83.72
RNN	After	0.7	0.8	0.00
RNN	After	0.7	0.9	50.00
RNN	After	1.0	0.8	60.00
RNN	After	1.0	0.9	74.42
LSTM1	Before	0.7	0.8	86.11
LSTM1	Before	0.7	0.9	83.72
LSTM1	Before	1.0	0.8	76.00
LSTM1	Before	1.0	0.9	76.00
LSTM1	After	0.7	0.8	86.05
LSTM1	After	0.7	0.9	88.10
LSTM1	After	1.0	0.8	71.43
LSTM1	After	1.0	0.9	75.68
LSTM2	Before	0.7	0.8	89.58
LSTM2	Before	0.7	0.9	84.62
LSTM2	Before	1.0	0.8	86.54
LSTM2	Before	1.0	0.9	76.47
LSTM2	After	0.7	0.8	**90.62**
LSTM2	After	0.7	0.9	89.36
LSTM2	After	1.0	0.8	88.64
LSTM2	After	1.0	0.9	87.50

The results in this appendix show that spelling accuracy improves across all models after fine-tuning. RNN shows inconsistent gains, with some severe degradation (e.g., 0% at T=0.7, p=0.8), while both LSTM1 and LSTM2 consistently improve. The highest score (90.62%) was achieved by LSTM2 after fine-tuning at T=0.7, p=0.8. This suggests that deeper models not only learn better structure, but also generate more syntactically valid and coherent code tokens.